Using Objects

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **construct** an object with `new`
2. Be able to **distinguish** `static` class members from instance members.
3. Be able to **debug** a `NullPointerException` by naming the null reference.
4. Be able to **explain** why objects are compared with `.equals()`, not `==`.

What is an object?

Definition: an object bundles **state** — the data it remembers — with **behavior** — the methods you can call on it.

A `String` remembers a sequence of characters, and knows how to hand you `.length()`, `.charAt(0)`, or an upper-cased copy of itself with `.toUpperCase()`.

These are all **instance methods**.

`new` builds the object

Strings *are* objects, but get special privileges, i.e...

```
String str1 = new String("hello"); // or, more simply...
String str2 = "world";
```

Objects are created via `new` ...

```
Book dune = new Book("Dune");
```

Three things happen, right to left: `new` reserves the space, the **constructor** `Book("Dune")` fills in the starting state, and the **reference** lands in `dune`.

A class can offer more than one constructor

We don't cover *creating* classes until Week 4, but a quick sneak peek inside can be helpful...

```
public Book(String bookTitle)                    // arrives on the shelf
public Book(String bookTitle, boolean startsOut)    // arrives already on loan
```

Same name, different **parameter lists** – that is **overloading**. Java picks by the arguments you pass, so `new Book("Hamlet", true)` runs the second one.

An object is an instance of a class

A class is a "recipe" for an object.

Meet the Book

You will use classes you did not write every day – `String`, `Scanner`, `ArrayList`. Here is one more. Let's build one and watch its constructor run.

→ Step through it in Java Visualizer.

Fields belong to the object

```
private String title;           // one title per Book
private boolean checkedOut;     // one status per Book
```

These are **instance fields**. They hold the **state** of the instantiated object. Every `new Book(...)` brings along a fresh set of them, and nothing else in the program can see that set except through the object.

Tophat Q1

A quick check on instances.

→ Open Tophat to vote.

Some state has no object to live in

How long is a loan? That is a library policy, not a specific fact about `dune`. Copying it into every `Book` would be inefficient; it belongs to the **class** rather than each instance.

```
private String title;                 // INSTANCE field: one per Book
public static int LOAN_DAYS = 14;     // CLASS field: one, for all Books
```

→ Step through it in Java Visualizer

Who does the method belong to?

	Class (`static`)	**Instance**
Called on	the class name	a reference
Example	`Math.pow(2, 3)`	`name.toUpperCase()`

Read the signature, then write the call

Same question, now for methods. Most classes arrive compiled: you get the signatures and nothing else.

```
class Math
    static int max(int a, int b)
class Book
    static void setMaxLoanDays(int days)
    void checkOut()
    String getTitle()
```

Which of these need `Book` in front of them?

Which of these need a `Book` *object* in front of them?

The error you will actually see

```
public class Main {
    private int count = 0;       // instance field

    public static void main(String[] args) {
        count++;                 // will not compile
    }
}
```

"Cannot make a static reference to the non-static field count." `main` is `static`, so it runs with no object — there is no `count` to reach.

`count` *must* be `static` for this to work.

Tophat Q2

A quick check on instance vs static.

→ Open Tophat to vote.

`null` is the absence of an object

```
Book[] shelf = new Book[3];   // 3 slots, all null
```

That builds **one** array object. It does *not* build 3 Books – every slot starts empty, and filling them is your job.

Watch what an unguarded loop does to it.

→ Step through it in Java Visualizer.

Guard before you dereference

```
for (int i = 0; i < shelf.length; i++) {
    if (shelf[i] != null) {
        System.out.println(shelf[i].getTitle());
    }
}
```

Any array that can hold gaps needs this shape. We should **guard** against possible `null` references.

The same is true *anywhere* that *could* be `null`.

Tophat Q2

A quick check on `null` and reference parameters.

→ Open Tophat to vote.

Wrapper classes

Reminder: primitives are held by a variable directly; objects are held by their memory address.

A primitive variable **cannot** hold null. The best we can do is assign a **sentinel value** like `-1`.

In some cases, we'd prefer `null`. Use **wrappers!**

A **wrapper class** is an object whose whole job is to hold one primitive value, or `null`.

Wrapper classes

A **wrapper class** is an object whose whole job is to hold one primitive value, or `null`.

Primitive	Wrapper
`int`	`Integer`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

How do these compare in memory?

One is held, the other is pointed at.

```
int a = 256;
Integer b = new Integer(256);
```

→ Step through it in Java Visualizer, but enable the "Show String/Integer/etc objects, not just values"!.

Also consider, does `a == b`?

Autoboxing and unboxing

Values are auto-boxed and auto-unboxed

```
int x = 5;
Integer y = x;  // autoboxing
```

```
Integer x = 5;
int y = x;      // auto-unboxing
```

`Integer == int` leads to *unboxing*, so the prior slide's comparison is `true`.

Autoboxing and unboxing

What about the following code?...

```
int x = 256;
int y = 256;
```

```
Integer a = 256;
Integer b = 256;
```

Does `x == y`? Does `a == b`? Try them out!

→ Visualize x vs y

→ Visualize a vs b

Autoboxing and unboxing

Doing an `int` to `int` comparison, what are we comparing? Values! → `true`

Doing an `Integer` to `Integer` comparison, what are we comparing? Memory addresses! → `false` *

Integer Comparison

```
Integer a = 127, b = 127;
System.out.println(a == b);

Integer c = 128, d = 128;
System.out.println(c == d);
```

Two identical-looking comparisons, two different answers. Why? Caching of small `Integer`

→ Step through it in Java Visualizer.

`==` asks "same object?"

Java caches small `Integer` objects (-128 through 127), so `a` and `b` really *are* one object. At 128 you get two objects with equal contents — and `==` says false.

Rule: compare objects (e.g. `String`, `Integer`, anything!) with `.equals()`. Save `==` for primitives or *reference comparison*.

Wrappers and Strings are immutable

Immutable: *cannot* be changed!

We can re-assign to a new object, but we cannot change the existing one.

```
String name = "ada";
name.toUpperCase();           // returns "ADA" -- and throws it away
name = name.toUpperCase();    // this is what you meant
```

Nothing can change these objects after they are built, so every "changing" method hands back a **new** one.

Tophat Q3

→ Open Tophat to vote.

Eclipse

More tips & auto-generating JavaDocs.

Questions?